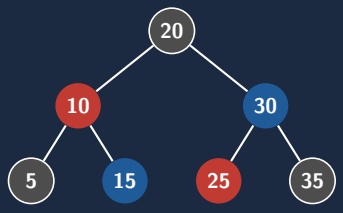
ALGORITMOS E ESTRUTURAS DE DADOS II

Sistema Bancário com Árvores AVL e Rubro-Negra

Implementação e comparação experimental de desempenho



Universidade Federal do Amazonas (UFAM)

Instituto de Ciências Exatas e Tecnologia (ICET) · Engenharia de Software

Professor: Alternei Brito

Equipe

I1	Luiz Gustavo	Árvore AVL, cenários de inserção A e B, revisão da Rubro-Negra e do modelo
I2	Julio Augusto	Árvore Rubro-Negra, análise das recolorações, revisão da aplicação e dos experimentos
I3	Adriano Monteiro	Modelo, aplicação, menu, gerador, experimentos C e D, consolidação; revisão da AVL

Em uma frase. Com os mesmos dados nas duas árvores, a **AVL** terminou mais baixa na inserção ordenada (altura **13** contra **23** para $N = 10.000$) e empatou na aleatória (**15** $\times$ **15**); a **Rubro-Negra** foi mais rápida em todas as inserções e remoções medidas, e a **AVL** foi mais rápida nas buscas.

Itacoatiara — AM, outubro de 2026

Sumário

1 Identificação e objetivo **4**

2 Minimundo e modelo de dados **4**

3 Arquitetura do sistema **6**

4 Convenções adotadas **7**

5 Implementação da árvore AVL **7**

5.1 Nó, altura e fator de balanceamento . . . 7
5.2 Busca . . . 8
5.3 Rotações . . . 8
5.4 Os quatro casos de rebalanceamento . . . 9
5.5 Inserção . . . 9
5.6 Remoção . . . 10
5.7 Validação . . . 11
5.8 Exibição, estatísticas e modo detalhado . . . 11
5.9 Testes . . . 12
5.10 Cenários A e B . . . 12

6 Implementação da árvore Rubro-Negra **12**

6.1 Nó e sentinela NIL . . . 13
6.2 Busca e inserção . . . 13
6.3 Remoção . . . 14
6.4 Contagem de rotações e recolorações . . . 15
6.5 Validação . . . 15
6.6 Testes . . . 15
6.7 Análise das recolorações . . . 16
6.7.1 De onde vêm as recolorações na inserção . . . 16
6.7.2 De onde vêm as recolorações na remoção . . . 17
6.7.3 Relação com a AVL . . . 17

7 Metodologia **18**

7.1 Parâmetros . . . 18
7.2 Regras de justiça . . . 18
7.3 Como os resultados são apresentados . . . 19

8 Resultados **20**
8.1 Comparação direta ($N = 10.000$) . . . 20
8.2 Inserção: altura, tempo e rotações . . . 21
8.3 Busca . . . 22
8.4 Remoção . . . 23

9 Discussão: respostas às perguntas do enunciado **23**
9.1 Qual estrutura apresentou menor altura? . . . 23
9.2 Qual estrutura realizou mais rotações? . . . 24
9.3 Como as recolorações da Rubro-Negra aparecem nos experimentos? . . . 24
9.4 A menor altura resultou sempre em menor tempo de execução? . . . 24
9.5 Como a ordem de inserção influenciou as duas estruturas? . . . 24
9.6 Em quais operações as diferenças foram mais evidentes? . . . 25
9.7 Quais limitações metodológicas existem? . . . 25

10 Conclusão **26**

A Resultados completos **27**

B Testes e reprodução **28**
B.1 Suíte de testes . . . 28
B.2 Como reproduzir . . . 28

Como ler este relatório Todos os números foram calculados a partir dos arquivos de `resultados/` produzidos pela coleta oficial de 24/10/2026, 23:46. Quadros **verdes** trazem resultados medidos; quadros **âmbar** trazem expectativas teóricas, que *não* foram medidas e não devem ser lidas como resultado. Em gráficos e tabelas, a **AVL** aparece sempre em azul com círculos e a **Rubro-Negra** em vermelho com quadrados.

1 Identificação e objetivo

Este relatório documenta o trabalho prático de AED II: um sistema bancário em memória cujas contas são armazenadas, ao mesmo tempo, em uma árvore **AVL** e em uma árvore **Rubro-Negra**, ambas implementadas pela equipe, sem bibliotecas de árvore. O código-fonte, os testes, os resultados brutos e as instruções de execução estão no repositório **AED2_sistema_bancario** (veja o `README.md`).

Objetivo. Implementar as duas árvores com as mesmas convenções de contagem e comparar, de forma reprodutível, o custo de inserção, busca e remoção em quatro cenários, observando tempo de execução, altura, rotações, recolorações e nós visitados. As conclusões ficam restritas ao que os dados medidos sustentam.

Tabela 1. Divisão do trabalho.

Integrante	Responsabilidade principal	Revisou
I1 — **Luiz Gustavo**	`avl.py`; cenários A e B (`experimentos_insercao.py`); demonstração (`demo_avl.py`)	`rubro_negra.py`, `modelos.py`
I2 — **Julio Augusto**	`rubro_negra.py`; análise das recolorações	aplicação e experimentos
I3 — **Adriano Monteiro**	`modelos.py`, `aplicacao.py`, `main.py`, `gerador.py`, `experimentos.py`, `gerar_graficos.py`, `README.md`	`avl.py`

2 Minimundo e modelo de dados

O sistema representa o cadastro de contas de um banco. A entidade única é a **Conta**, definida em `modelos.py` como um `dataclass` *imutável* (`frozen=True`) que valida e normaliza todos os campos na criação. Como é imutável, as duas árvores podem guardar o *mesmo* objeto sem que uma consiga alterar a conta da outra.

Tabela 2. Campos da conta e regras de validação.

Campo	Tipo	Regra	Exemplo de normalização
`numero`	`int`	1 a 9 dígitos, maior que zero; sem sinal nem espaço interno. **Chave** das árvores	`"000123 "` → `123`
`titular`	`str`	2 a 100 caracteres; espaços repetidos colapsados	`"Ana  Silva "` → `"Ana Silva"`
`agencia`	`str`	1 a 4 dígitos, completada com zeros à esquerda	`"12"` → `"0012"`
`tipo`	`str`	`CORRENTE` ou `POUPANCA`; aceita minúsculas e acentos	`"poupança"` → `"POUPANCA"`
`saldo`	`Decimal`	não negativo, até 2 casas, vírgula ou ponto; guardado com 2 casas	`"1500,5"` → `1500.50`

Os valores monetários usam `Decimal`, e não `float`, para evitar erros de arredondamento binário. Entradas inválidas levantam `ValueError` com mensagem específica; são rejeitados, por exemplo, `+123`, `12 3`, `1_000` e números com mais de 9 dígitos no campo `numero`, e `1e2`, `-0`, `1.234,56` e `0,001` no saldo.

Tabela 3. Comportamento nas situações-limite (igual nas duas árvores).

Situação	Comportamento
Número já cadastrado	rejeitado: `inserir` devolve `False` e nada muda; o menu avisa a duplicata
Busca ou remoção de número inexistente	devolve `None`; a árvore e os contadores de rotação não mudam
Árvore vazia	consultas respondem com segurança: `None`, lista vazia, altura -1, tamanho 0
AVL e Rubro-Negra divergem	a aplicação interrompe a operação com `InconsistenciaEntreArvores` (ERRO CRÍTICO no menu)

3 Arquitetura do sistema

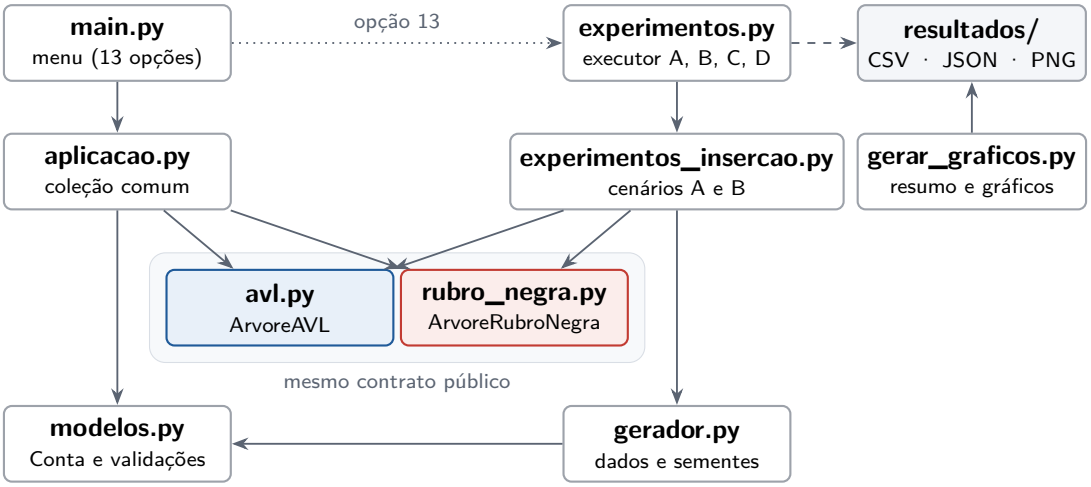


Figura 1. Módulos e dependências. Setas cheias: "usa"; tracejada: "grava". O executor também cria as árvores e usa o gerador diretamente para os cenários C e D (ligações omitidas para clareza).

Tabela 4. Responsabilidade de cada módulo.

Arquivo	Função
`modelos.py`	classe `Conta`, funções de normalização e `criar_conta()`
`avl.py`	`ArvoreAVL`: inserção, busca, remoção, consultas, caminhamentos, exibição, validação e estatísticas
`rubro_negra.py`	`ArvoreRubroNegra`: mesmo contrato, com sentinela NIL e implementação iterativa (estilo CLRS)
`aplicacao.py`	`AplicacaoBancaria`: cadastro e remoção nas duas árvores, comparando os retornos; consultas na árvore ativa
`main.py`	menu de 13 opções; repassa as entradas como texto, para que toda validação ocorra em `modelos.py`
`gerador.py`	dados determinísticos (`chaves_crescentes`, `permutacao`, `gerar_contas`, `buscas`, `subconjunto_remocao`) e as sementes oficiais
`experimentos_insercao.py`	cenários A e B, com as regras de justiça; chamado por `experimentos.py`
`experimentos.py`	executor oficial: roda A, B, C e D e grava os CSVs e a configuração
`gerar_graficos.py`	resumo estatístico (`resumo_medias.csv`) e gráficos PNG
`demo_avl.py`	demonstração passo a passo da AVL, usada no ensaio da defesa
`testes/`	158 testes automatizados (`unittest`), detalhados no Apêndice B

4 Convenções adotadas

Para que as métricas das duas árvores sejam comparáveis, o grupo fixou as regras abaixo antes de codificar; elas valem igualmente em `avl.py` e `rubro_negra.py`.

Tabela 5. Convenções comuns.

Chave	apenas `conta.numero` participa da ordenação
Altura	em *arestas*: árvore vazia vale -1 e folha vale 0; o NIL não conta
Nós visitados	contados somente em `buscar()`: cada nó *real* cuja chave foi comparada soma 1
Rotações	simples conta 1; dupla conta 2 (AVL: casos LR e RL; Rubro-Negra: caso triângulo)
Recolorações	só na Rubro-Negra, e só quando a cor realmente muda; na AVL a métrica "não se aplica"
Remoção com 2 filhos	o *sucessor* (mínimo da subárvore direita) assume o lugar, levando a conta inteira
Validação	`validar()` recalcula as propriedades do zero, sem confiar em campos guardados
Contrato	`inserir`, `buscar`, `remover`, `minimo`, `maximo`, três caminhamentos, `altura`, `tamanho`, `exibir`, `validar`, `estatisticas`, `resetar_estatisticas`, `definir_modo_detalhado`

5 Implementação da árvore AVL

> **Responsável:** I1 — Luiz Gustavo · **Revisor:** I3 — Adriano Monteiro · `avl.py`, `testes/test_avl.py` (50 testes)

A AVL está em `avl.py` (classe `ArvoreAVL`). Inserção e remoção são *recursivas*: a recursão desce até o ponto da operação e, na volta, cada ancestral recalcula a própria altura e se rebalanceia. A busca é iterativa. Só a biblioteca padrão do Python é usada.

5.1 Nó, altura e fator de balanceamento

Cada nó (`_NoAVL`, com `__slots__`) guarda a **conta inteira**, os dois filhos e a **altura** da própria subárvore. O fator de balanceamento não é armazenado: é calculado quando necessário,

$$\mathrm{FB}(v) \;=\; h(\text{esquerda}) - h(\text{direita}), \qquad h(\varnothing) = -1.$$

Guardar a altura, e não o FB, simplifica a manutenção: a altura de qualquer nó se recalcula a partir dos filhos com uma única fórmula, $h(v) = 1 + \max\{h(\text{esq}),\, h(\text{dir})\}$, enquanto o novo FB depois de uma rotação teria de ser deduzido caso a caso. A convenção $h(\varnothing) = -1$ faz uma folha ter altura $1 + \max(-1, -1) = 0$ sem nenhum caso especial. Um nó é válido quando $|\mathrm{FB}| \le 1$.

5.2 Busca

A busca é iterativa: compara a chave procurada com a do nó atual e desce à esquerda ou à direita até encontrá-la ou chegar a um filho vazio. Cada nó real comparado soma 1 em `nos_visitados` e cada chamada soma 1 em `buscas`. Inserção e remoção *não* passam por `buscar()` e, por isso, não alteram esse contador (convenção da Tabela 5).

5.3 Rotações

Uma rotação troca três ponteiros e preserva a ordem de busca: a subárvore B muda de pai, mas continua entre x e y.

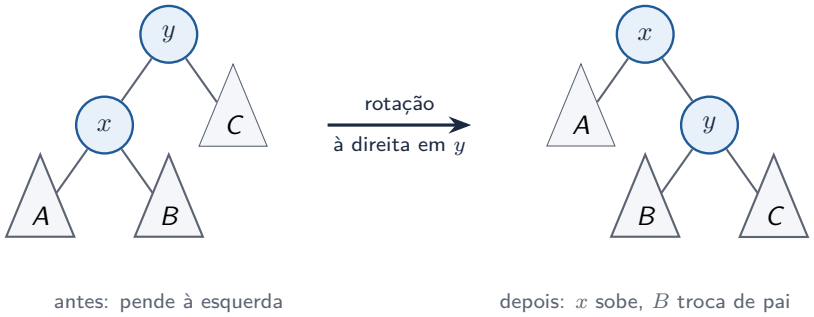


Figura 2. Rotação simples à direita. A rotação à esquerda é o espelho.

Depois da troca, a altura de y é atualizada *antes* da de x, porque y virou filho de x e a altura do pai depende da do filho. Cada chamada de `_rotacao_direita` ou `_rotacao_esquerda` soma 1 ao contador; uma rotação dupla é a composição de duas chamadas e, portanto, soma 2 sem nenhuma lógica adicional.

5.4 Os quatro casos de rebalanceamento

O método `_balancear` é chamado em cada ancestral na volta da recursão. Ele atualiza a altura e, se $|\mathrm{FB}| = 2$, escolhe o caso pelo sinal do FB do filho mais alto:

Tabela 6. Casos de rebalanceamento da AVL.

FB do nó	FB do filho mais alto	Caso	Ação	Rotações
$+2$	≥ 0	LL	rotação à direita no nó	1
$+2$	< 0	LR	à esquerda no filho, depois à direita no nó	2
-2	≤ 0	RR	rotação à esquerda no nó	1
-2	> 0	RL	à direita no filho, depois à esquerda no nó	2

O uso de ≥ 0 (e não > 0) no caso LL, e de ≤ 0 no RR, cobre o filho com $\mathrm{FB} = 0$. Esse caso nunca ocorre na inserção, mas ocorre na remoção, e a resposta correta é a rotação *simples*; tratá-lo como rotação dupla é um erro clássico, que os testes da Seção 5.9 detectam.

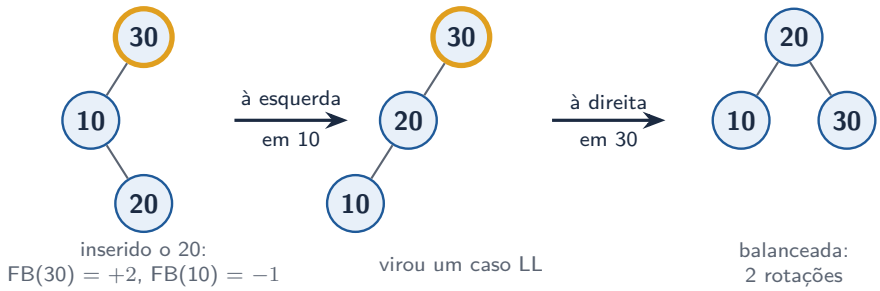


Figura 3. Caso LR com as chaves 30, 10 e 20. O nó desbalanceado está destacado.

Saída real do modo detalhado — inserção de 30, 10, 20

```
[AVL] inserido 30 como raiz
[AVL] inserido 10 sob 30
[AVL] inserido 20 sob 10
[AVL] desbalanceado em 30 (FB=+2) → caso LR
[AVL] rotação à esquerda em 10 → 20 sobe
[AVL] rotação à direita em 30 → 20 sobe
```

5.5 Inserção

A inserção desce recursivamente comparando as chaves, cria a folha no ponto vazio e, na volta, cada nível chama `_balancear` e *devolve a nova raiz da sua subárvore*. Por isso o pai pode simplesmente escrever `no.esquerda = self._inserir(no.esquerda, ...)`: se houve rotação lá embaixo, ele já recebe o ponteiro atualizado.

Quando a chave já existe, uma marca interna fica falsa e todos os níveis acima devolvem o nó intacto, sem recalcular nada; assim a duplicata nunca gera rotação e a conta original não é sobrescrita. Na inserção basta uma correção (simples ou dupla): a rotação devolve a subárvore à altura que ela tinha antes da inserção, e os ancestrais acima não percebem mudança.

5.6 Remoção

A remoção também é recursiva e trata três casos:

- **Folha** ou **um filho**: o filho (ou o vazio) sobe para o lugar do nó;
- **Dois filhos**: o *sucessor* (mínimo da subárvore direita) assume o lugar, com a `Conta` inteira (titular, agência, tipo e saldo), e depois é removido lá de baixo. O sucessor nunca tem filho esquerdo, então essa segunda remoção sempre cai no caso folha ou um filho. A conta original é guardada *antes* da substituição e é ela que `remover()` devolve.

A diferença em relação à inserção está na volta da recursão: uma rotação na remoção pode *diminuir* a altura da subárvore, o que desequilibra o nível de cima. Por isso a remoção pode precisar de correções em vários ancestrais, até a raiz. Como a recursão passa por todos eles, isso acontece sem código adicional. Chave inexistente não altera a árvore, o tamanho nem os contadores.

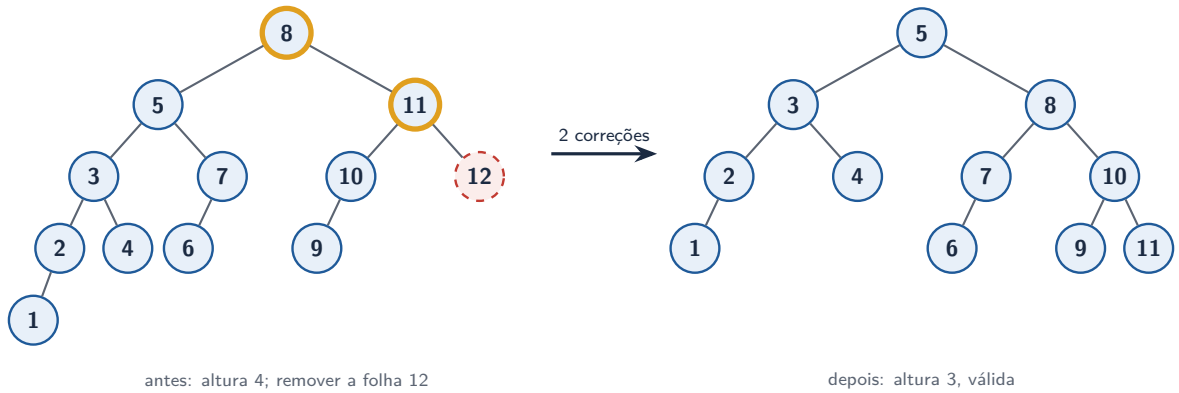


Figura 4. Remoção com correção em dois níveis. A árvore de Fibonacci de altura 4 é a AVL com o menor número de nós possível para essa altura (12): em cada nó interno um lado é exatamente um nível mais alto.

Saída real do modo detalhado — remoção do 12

```
[AVL] removido 12 (folha)
[AVL] desbalanceado em 11 (FB=+2) → caso LL
[AVL] rotação à direita em 11 → 10 sobe
[AVL] desbalanceado em 8 (FB=+2) → caso LL
[AVL] rotação à direita em 8 → 5 sobe
```

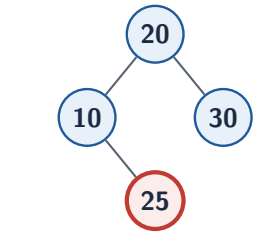
A primeira rotação corrige o 11, mas encolhe aquela subárvore; com isso a raiz 8 fica com FB $= +2$ e também precisa girar. No caso do plano de testes (inserir 4, 2, 6, 1, 3 e remover 6), o 4 fica com FB $= +2$ e o filho 2 com FB $= 0$: é o caso que só existe na remoção, resolvido com uma rotação simples (pré-ordem final 2, 1, 4, 3).

5.7 Validação

`validar()` devolve `(True, [])` ou `(False, [violações])` e **não confia em nada que a árvore guarda**: recalcula tudo numa única passada recursiva que devolve, para cada subárvore, a altura real e a quantidade de nós. Para cada nó, confere:

1. **Ordem de busca com limites herdados.** Cada nó recebe um intervalo $(\min, \max)$ acumulado de *todos* os ancestrais: ao descer para a esquerda, o máximo passa a ser a chave atual; para a direita, o mínimo. Comparar só pai e filho deixaria passar o erro da Figura 5.
2. $|\mathrm{FB}| \leq 1$, com o FB calculado pelas alturas *reais* dos filhos. Se usasse o campo armazenado, um defeito que gravasse alturas erradas poderia esconder um desbalanceamento.
3. **Altura armazenada igual à recalculada.**

Ao final, confere se o tamanho armazenado é igual ao número real de nós. O validador não para no primeiro erro: devolve a lista completa, com mensagens como as do exemplo.



O 25 é maior que o pai 10, então a comparação local parece correta. Mas ele está à *esquerda* do 20, e portanto deveria ser menor que 20. Com os limites herdados a violação aparece:

```
ordem violada:  25 fora do intervalo permitido (10, 20)
```

Figura 5. Por que a ordem precisa ser conferida com limites herdados.

5.8 Exibição, estatísticas e modo detalhado

`exibir()` *devolve* (não imprime) a árvore deitada, com a raiz à esquerda e o filho direito acima, mostrando cada nó como `numero [h=altura, FB=fator]`; quem decide quando imprimir é o menu. `estatisticas()` devolve rotações, nós visitados e buscas desde o último `resetar_estatisticas()`; recolorações valem `None` ("não se aplica"). Como recurso extra, `rotacoes_por_caso()` informa quantos rebalanceamentos de cada caso ocorreram, com $\mathrm{LL} + \mathrm{RR} + 2(\mathrm{LR} + \mathrm{RL})$ igual ao total de rotações; é dela que vem a Tabela 15 da Seção 8.

Detalhe de medição. No modo detalhado, cada passo é impresso por `_log`. Em Python, a *f-string* de uma mensagem é montada *antes* da chamada; por isso, mesmo com o modo desligado, o texto era construído e descartado dentro do intervalo cronometrado. Numa medição feita durante o desenvolvimento (inserção de 10.000 contas, ambiente Linux), esse custo extra foi de 6 % a 10 % na AVL e de 36 % a 43 % na Rubro-Negra, que gera mais mensagens por operação. Como o efeito é desigual, distorceria a comparação. A correção, aplicada nas duas árvores antes da coleta oficial, foi pôr toda chamada de log dentro de `if self._detalhado:`; com ela, o custo extra medido caiu para cerca de 0 %.

5.9 Testes

`testes/test_avl.py` tem 50 testes. Os valores esperados dos contadores foram calculados à mão antes da execução. Uma função própria (`verificar_avl`) confere ordem, |FB|, alturas e tamanho *sem usar* `validar()`, para que um defeito na árvore não esconda outro no teste.

Tabela 7. Testes da AVL por grupo.

Grupo	Testes	O que verifica
Rotações	6	LL, RR (1 rotação), LR, RL (2 rotações), sem rotação, rotação num nó interno
Inserção	5	duplicata não sobrescreve nem rotaciona; 1..1023 crescente gera árvore perfeita de altura 9; 2.000 inserções contra um conjunto de referência
Busca e consultas	6	contagem de nós visitados; reset; árvore vazia, um nó, caminhamentos, mínimo e máximo
Remoção	11	caso do plano; Fibonacci (dois níveis); folha, 1 filho, 2 filhos com atributos preservados; raiz e último nó; chave inexistente; 2.000 operações mistas contra um `dict`, validando a cada passo
Validação	10	árvores válidas e 6 tipos de corrupção montados à mão: ordem violada só no neto, chave igual a ancestral, desbalanceamento, altura errada, FB escondido por alturas falsas, tamanho errado
Exibição	5	formato exato, barras ligando os nós, retorno sem imprimir
Casos e modo detalhado	7	`rotacoes_por_caso()` e a soma igual ao total; modo detalhado produz a mesma árvore

Para confirmar que os testes detectam erros de verdade, foram introduzidas **sete falhas propositais** (não rebalancear no caminho do sucessor; não copiar o sucessor; não decrementar o tamanho; tratar FB = 0 como rotação dupla; validar só pai e filho; calcular o FB pelas alturas armazenadas; aceitar chave repetida). Todas fizeram a suíte falhar.

5.10 Cenários A e B

O módulo `experimentos_insercao.py` implementa os cenários de inserção e é chamado pelo executor oficial. Para cada combinação de cenário, N e repetição, as contas são geradas *uma vez* e os *mesmos objetos* são entregues às duas árvores; cada árvore é nova, com contadores zerados e modo detalhado desligado. Só o laço de inserções fica dentro do cronômetro; a validação vem depois. Os testes do módulo (12) usam uma árvore "espiã" para provar que as duas recebem os mesmos objetos na mesma ordem e um relógio interceptado para provar que a validação nunca entra no cronômetro.

6 Implementação da árvore Rubro-Negra

> **Responsável:** I2 — Julio Augusto · **Revisor:** I1 — Luiz Gustavo · `rubro_negra.py`, `testes/test_rubro_negra.py` (66 testes)

A Rubro-Negra está em `rubro_negra.py` (classe `ArvoreRubroNegra`), com a mesma interface da AVL. A implementação é *iterativa*, com ponteiro para o pai em cada nó, seguindo o modelo de Cormen et al. (CLRS). Usa apenas a biblioteca padrão do Python.

6.1 Nó e sentinela NIL

Cada nó (`_NoRN`) guarda a conta inteira, a cor (V ou P), os filhos esquerdo e direito e o pai. Em vez de `None`, todo filho ausente e o pai da raiz apontam para um único nó **sentinela NIL** por árvore, sempre preto e sem conta. Com o sentinela, a cor de um filho ausente pode ser lida diretamente (o NIL conta como preto), e as rotações e a remoção não precisam de testes especiais para `None`. O NIL nunca é retornado, não aparece nos caminhamentos nem na exibição e não entra na altura.

6.2 Busca e inserção

A busca é iterativa: compara a chave procurada com a do nó e desce até encontrá-la ou chegar ao NIL. Cada nó real comparado soma 1 em `nos_visitados`; o NIL não conta.

A inserção desce como numa árvore binária de busca até a posição do novo nó. Chave repetida é rejeitada: o método devolve `False` sem alterar a árvore nem os contadores. O nó novo nasce **vermelho**, com os dois filhos apontando para o NIL. Se o pai dele também for vermelho, há uma violação vermelho-vermelho, corrigida em laço. Com o pai à esquerda do avô (o lado direito é o espelho):

Tabela 8. Casos da correção após a inserção.

Caso	Condição	Ação	Rotações	Recolorações
Tio vermelho	pai e tio vermelhos	pai e tio ficam pretos, avô fica vermelho; a correção continua a partir do avô	0	3
Triângulo	tio preto, nó à direita do pai	rotação no pai transforma no caso linha	2 (com a da linha)	2
Linha	tio preto, nó à esquerda do pai	pai fica preto, avô fica vermelho, rotação no avô; termina	1	2

Ao final, a raiz é pintada de preto. Os casos triângulo e linha encerram a correção; por isso uma inserção faz no máximo 2 rotações, e só o caso tio vermelho pode se repetir subindo até a raiz.

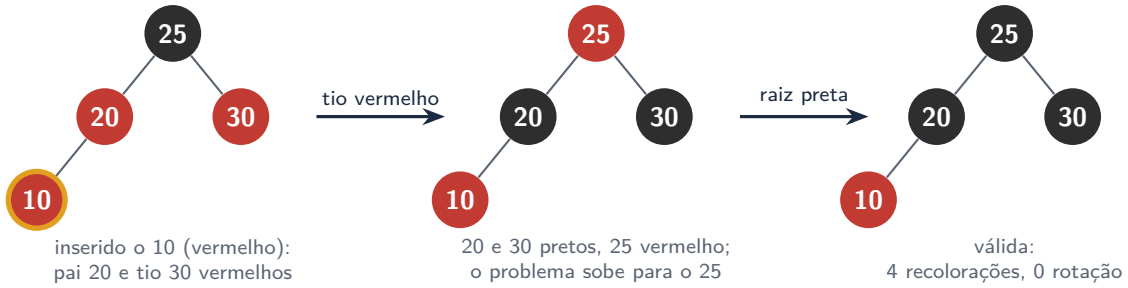


Figura 6. Caso tio vermelho propagando até a raiz (inserção de 25, 20, 30 e 10).

Saída real do modo detalhado — inserção do 10

```
[RN] inserido 10 sob 20
[RN] tio 30 vermelho → recolorir 20(P), 30(P), 25(V)
[RN] recolorações: 4
[RN] raiz 25 pintada de preto (recolorações: 5)
```

O contador acumulado chega a 5 porque a primeira inserção (25) já havia custado 1 recoloração: o nó nasce vermelho e é pintado de preto ao virar raiz.

6.3 Remoção

A remoção usa `_transplantar(u, v)`, que coloca a subárvore v no lugar de u e atualiza `v.pai`, mesmo quando v é o NIL; a correção usa esse ponteiro para saber onde está o problema.

- **Folha ou um filho:** o filho (ou o NIL) sobe para o lugar do nó;
- **Dois filhos:** o sucessor (mínimo da subárvore direita) sai da sua posição e assume o lugar do nó removido, herdando a cor dele. O nó do sucessor se move inteiro, então a `Conta` vai junto, com todos os atributos intactos.

Se a cor que saiu da árvore era preta, um dos caminhos perdeu um nó preto. O nó x que ocupou o lugar (que pode ser o NIL) passa a carregar um "preto duplo", eliminado em laço. Com x à esquerda do pai e w como irmão (o lado direito é o espelho):

Tabela 9. Casos da correção após a remoção.

Caso	Condição	Ação	Rotações
1	irmão vermelho	irmão fica preto, pai fica vermelho, rotação no pai; vira o caso 2, 3 ou 4	1
2	irmão preto, filhos dele pretos	irmão fica vermelho; o problema sobe para o pai	0
3	irmão preto, sobrinho próximo vermelho e distante preto	sobrinho próximo fica preto, irmão fica vermelho, rotação no irmão; vira o caso 4	1
4	irmão preto, sobrinho distante vermelho	irmão herda a cor do pai, pai e sobrinho distante ficam pretos, rotação no pai; termina	1

O caso 2 termina quando o pai é vermelho (ele é pintado de preto) ou quando o problema chega à raiz. No final, x é pintado de preto. Como os casos 1, 3 e 4 levam ao fim da correção em poucos passos, uma remoção faz no máximo 3 rotações; só o caso 2 pode se repetir subindo.

6.4 Contagem de rotações e recolorações

Cada rotação simples soma 1 em `rotacoes`. O triângulo da inserção e a sequência caso 3 + caso 4 da remoção fazem duas rotações e somam 2, como a rotação dupla da AVL. Toda troca de cor passa por um único método, `_pintar(no, cor)`, que compara a cor atual com a nova e só incrementa `recoloracoes` se ela realmente mudar. Consequências da regra:

- o nó novo nascer vermelho não conta;
- pintar a raiz de preto conta; por isso inserir a primeira chave custa 1 recoloração;
- no caso 4, o irmão herdar a cor do pai só conta quando as cores são diferentes;
- na remoção com 2 filhos, o sucessor herdar a cor do nó removido só conta quando as cores são diferentes;
- no caso tio vermelho que propaga até a raiz, a raiz vira vermelha e volta a preta: **2 recolorações** nela (Figura 6).

Rotações, recolorações, nós visitados e buscas têm contadores separados, e `resetar_estatisticas()` zera todos sem mexer na árvore. Assim como na AVL, toda chamada de log fica dentro de `if self._detalhado:`.

6.5 Validação

`validar()` devolve `(True, [])` ou `(False, [violações])`, recalculando tudo a partir da raiz sem confiar nos contadores. Ela confere: NIL preto, raiz preta e pai da raiz igual ao NIL; ordem de busca com limites $(\min, \max)$ herdados de todos os ancestrais; nenhum nó vermelho com filho vermelho; mesma altura negra nos dois lados de cada nó, ou seja, em todos os caminhos até o NIL; ponteiro de pai de cada filho apontando para o nó certo; tamanho armazenado igual à contagem real.

6.6 Testes

`testes/test_rubro_negra.py` tem 66 testes, com os valores esperados dos contadores calculados à mão antes de rodar:

- **Casos do plano:** inserir 10 resulta em 1 recoloração e 0 rotação; inserir 10, 20, 30 resulta em 1 rotação e 3 recolorações; os dois triângulos resultam em 2 rotações e 3 recolorações.
- **Correções isoladas:** cada caso de correção da inserção e da remoção, com os espelhos, em árvores pequenas montadas nó a nó com as cores escolhidas.
- **Bordas:** árvore vazia, duplicata, chave inexistente, remoção da raiz e do último nó, sentinela de volta ao estado neutro.
- **Validador:** detecta árvores corrompidas de propósito (raiz vermelha, vermelho-vermelho, altura negra diferente, ordem violada em neto, pai inconsistente, NIL vermelho, tamanho errado).
- **Aleatório:** 2.000 operações de inserção, remoção e busca comparadas com um `dict`, validando a árvore a cada passo.

Os testes conferem as propriedades com uma verificação própria (`verificar_rn`), independente de `validar()`. Na revisão, foram introduzidas **seis falhas propositais** (não pintar o sobrinho

distante no caso 4; não pintar o irmão no caso 3; não pintar o irmão no caso 2; não herdar a cor no sucessor; não pintar o tio no caso tio vermelho; não recolorir no caso linha), e todas fizeram a suíte falhar.

6.7 Análise das recolorações

> **Análise:** I2 — Julio Augusto. Os valores foram recalculados a partir dos CSVs oficiais; a origem de cada recoloração foi contada instrumentando `_pintar` com as mesmas sementes da coleta.

A recoloração é a operação de rebalanceamento mais frequente da Rubro-Negra. Em todos os cenários com $N = 10.000$ houve de 4 a 5 recolorações por rotação. A ordem de inserção mais que dobra o custo por operação: 4,99 recolorações por inserção na ordem crescente, contra 2,31 na aleatória.

Tabela 10. Recolorações e rotações da Rubro-Negra por operação ($N = 10.000$, média de 5 repetições).

Cenário	Operações	Recolorações por operação	Rotações por operação	Recolorações por rotação
A — inserção crescente	10.000	4,99	1,00	5,0
B — inserção aleatória	10.000	2,31	0,58	4,0
D — remoção de 10 %	1.000	1,40	0,30	4,6
D — remoção de 30 %	3.000	1,63	0,35	4,7
D — remoção de 50 %	5.000	1,78	0,37	4,9

Nos tamanhos menores o padrão é o mesmo: 4,45 e 4,91 recolorações por inserção no cenário A ($N = 100$ e 1.000), 2,24 e 2,30 no cenário B.

De onde vêm as recolorações na inserção

Cada caso da inserção tem custo fixo: o tio vermelho troca 3 cores, a linha e o triângulo trocam 2, e pintar a raiz troca 1. Contando as ocorrências de cada caso, a soma $3 \times$ tio vermelho $+ 2 \times$ (linha + triângulo) + raiz fecha exatamente com o total de recolorações em todos os tamanhos.

Tabela 11. Ocorrências de cada caso da inserção ($N = 10.000$).

Caso	A: por inserção	B: por inserção	Recolorações por ocorrência	Rotações por ocorrência
Tio vermelho	1,00	0,51	3	0
Linha	1,00	0,19	2	1
Triângulo	0,00	0,19	2	2
Raiz pintada de preto	12 no total	8 no total	1	0

O caso tio vermelho responde por 60 % das recolorações da inserção crescente (29.913 de 49.877) e por 67 % das da aleatória (15.384 de 23.096,8). Ele não faz rotação, mas é o único caso que se repete subindo pela árvore.

Na inserção crescente, toda chave nova entra no extremo direito, então nunca ocorre triângulo (assim como a AVL só teve o caso RR nesse cenário, Tabela 15). Praticamente toda inserção precisa de correção: metade custa 2 recolorações (só o caso linha); um quarto custa 5 (um tio

vermelho antes da linha); um oitavo custa 8 (dois tios vermelhos), e assim por diante. Cada nível a mais de propagação acontece na metade das vezes do anterior, como o "vai um" de um contador binário; na média, isso dá um tio vermelho e uma linha por inserção: 5 recolorações e 1 rotação.

Na inserção aleatória, 37 % das inserções não precisam de correção nenhuma, porque o novo nó cai sob um pai preto. Outras 28 % resolvem com um único caso linha ou triângulo (2 recolorações). É essa parcela sem correção, inexistente no cenário A, que reduz o custo médio para 2,31.

De onde vêm as recolorações na remoção

Na remoção, mais de um terço das recolorações não vem do laço de correção, e sim de ajustes locais. Com 30 % das chaves removidas ($N = 10.000$, 3.000 remoções):

Tabela 12. Origem das recolorações na remoção de 30 % ($N = 10.000$, média de 5 repetições).

Origem	Recolorações	Parcela
Caso 4 (sobrinho distante vermelho)	1.583,0	32 %
Nó que ocupou o lugar pintado de preto no final	1.138,6	23 %
Sucessor herda a cor do nó removido	713,6	15 %
Caso 2 (irmão preto com filhos pretos)	630,2	13 %
Caso 3 (sobrinho próximo vermelho)	538,4	11 %
Caso 1 (irmão vermelho)	290,0	6 %
Total	**4.893,8**	100 %

O caso 4 custa em média 2,5 recolorações por ocorrência, dentro do intervalo de 1 a 3 que o caso permite. O custo por remoção sobe de 1,40 para 1,78 conforme o percentual removido aumenta, e os dados mostram duas mudanças que acompanham essa subida: o caso 2, que pode se repetir subindo, passa de 0,10 para 0,32 ocorrência por remoção (de 10 % para 50 %); e as remoções de folha preta, que sempre geram um preto duplo, passam de 14 % para 22 % das remoções.

Relação com a AVL

A AVL não tem operação equivalente: seu rebalanceamento aparece só como rotações. A Rubro-Negra fez menos rotações por operação na inserção aleatória (0,58 contra 0,70) e na remoção de 10 % e 30 % (0,30 contra 0,35; 0,35 contra 0,38); na de 50 %, as duas ficaram praticamente iguais (0,37 e 0,37). Em troca, fez de 4 a 5 recolorações por rotação.

> **Expectativa teórica — não medida** No código, uma recoloração altera um único campo do nó, enquanto uma rotação religa até seis ponteiros. A expectativa, portanto, é que cada recoloração custe menos que uma rotação. O experimento, porém, mede só o tempo total de cada lote, não o custo de cada operação; por isso os dados não permitem atribuir a diferença de tempo entre as árvores às recolorações.

7 Metodologia

> **Responsáveis:** I3 — Adriano Monteiro, com I1 — Luiz Gustavo · **Revisor:** I2 — Julio Augusto · `experimentos.py`, `experimentos_insercao.py`, `gerador.py`

Os experimentos são executados por `python experimentos.py` e comparam as duas árvores sobre os mesmos dados em quatro cenários.

Tabela 13. Cenários experimentais.

Cenário	O que é executado	Métricas
A — inserção crescente	insere as chaves 1..N em ordem; entrada ordenada, caso típico de desbalanceamento	tempo, altura final, rotações, recolorações
B — inserção aleatória	insere uma permutação embaralhada de 1..N	tempo, altura final, rotações, recolorações
C — busca	árvore montada como no cenário B; $2N$ buscas em ordem aleatória: as N chaves existentes e N ausentes (N+1..$2N$)	tempo total, nós visitados por busca, presentes × ausentes
D — remoção	árvore montada como no cenário B; remove 10 %, 30 % e 50 % das chaves, escolhidas ao acaso	tempo, altura e tamanho antes e depois, rotações, recolorações

7.1 Parâmetros

- **Tamanhos e repetições:** $N = 100$, 1.000 e 10.000; 5 repetições por combinação, numeradas de 1 a 5. São 180 medições em `resultados_brutos.csv`: 3 tamanhos × 5 repetições × 12 combinações de cenário, percentual e estrutura. O cenário C gera ainda 222.000 linhas em `buscas.csv`, uma por busca.
- **Sementes** (fonte única em `gerador.py`): `SEMENTE_BASE = 2026`. Em cada repetição r e tamanho N, a permutação, as contas e as buscas usam a semente $2026 + 1000\,r + N$; o subconjunto removido usa $2026 + 500.000 + 1000\,r + N + p$, em que p é o percentual. Toda a aleatoriedade usa geradores locais (`random.Random(semente)`), nunca o gerador global.
- **Ambiente:** CPython 3.13.2 em Windows-11-10.0.26200-SP0, processador "Intel64 Family 6 Model 140 Stepping 1, GenuineIntel" (identificação devolvida pelo sistema), coleta em 24/10/2026, 23:46 (`config_experimentos.json`).

7.2 Regras de justiça

1. **Mesmos dados.** Para cada cenário, tamanho e repetição, os dados são gerados *uma vez* e a mesma lista (os mesmos objetos `Conta`) é entregue às duas árvores.
2. **Árvores novas.** Cada medição usa árvores novas, com modo detalhado desligado. Em C e D, a base é montada primeiro e os contadores são zerados *depois* da montagem e *antes* do lote medido. Em D, a base é reconstruída do zero para cada percentual; as remoções de 10 %, 30 % e 50 % não se acumulam.

3. **Cronômetro limpo.** `time.perf_counter()` envolve só o laço de operações: nenhum `print`, log, validação, leitura de estatísticas ou gravação de arquivo fica dentro do intervalo.
4. **Visitas por busca fora do cronômetro.** No cenário C, o laço cronometrado contém apenas as buscas. As visitas de cada busca são registradas numa segunda passada, sem cronômetro; como as buscas não alteram a árvore, a contagem é idêntica, e o executor confere que a soma das visitas individuais é igual ao total da passada cronometrada.
5. **Ordem alternada.** Nas repetições pares, a AVL é medida antes da Rubro-Negra; nas ímpares, depois. Isso dilui efeitos de aquecimento e de cache.
6. **Aquecimento.** Uma rodada completa de inserções, buscas e remoções é executada e descartada antes da coleta.
7. **Validação depois.** Ao fim de cada medição, as árvores são conferidas por `validar()`, e o tamanho final é comparado com o esperado. Uma falha marcaria a linha como inválida e a tiraria da análise.

> **Validade** As 180 medições terminaram com árvores válidas. Além disso, todas as métricas determinísticas (altura, rotações, recolorações e nós visitados) foram reproduzidas de forma independente, executando o código com as mesmas sementes em outro computador: as 180 linhas coincidiram *exatamente* com o CSV oficial. Só o tempo depende da máquina.

7.3 Como os resultados são apresentados

Os valores são médias das 5 repetições; o desvio-padrão amostral do tempo aparece entre parênteses no Apêndice A. Os tempos são gravados em segundos e apresentados em milissegundos. Os gráficos de tempo usam escalas logarítmicas nos dois eixos, porque N e o tempo variam em duas ordens de grandeza.

8 Resultados

> **Consolidação:** I3 — Adriano Monteiro · todos os valores são médias de 5 repetições, calculadas a partir de `resultados/resultados_brutos.csv` e `resultados/buscas.csv`.

8.1 Comparação direta ($N = 10.000$)

A Tabela 14 reúne as oito métricas comparativas exigidas para o maior tamanho testado. A tabela completa, com todos os tamanhos, cenários, percentuais e desvios, está no Apêndice A.

Tabela 14. Comparação AVL × Rubro-Negra para $N = 10.000$ (média de 5 repetições). A última coluna indica a estrutura com o menor valor.

#	Métrica	Cenário	AVL	Rubro-Negra	Menor
1	Altura final (arestas)	A	**13**	23	AVL
		B	15	15	empate
2	Rotações na inserção	A	9.986	9.976	RN (por 10)
		B	6.986,6	**5.788,8**	RN
3	Rotações na remoção (30 % das chaves)	D	1.144,2	**1.048**	RN
4	Recolorações	A (inserção)	não se aplica	49.877	—
		B (inserção)	não se aplica	23.096,8	—
		D 30 % (remoção)	não se aplica	4.893,8	—
5	Nós visitados por busca	C	13,19	**12,70**	RN
6	Tempo de inserção (ms)	A	51,12	**24,89**	RN
		B	47,28	**15,56**	RN
7	Tempo de busca, $2N$ buscas (ms)	C	**23,02**	24,71	AVL
8	Tempo de remoção, 30 % das chaves (ms)	D	17,72	**4,72**	RN

O desvio-padrão dos tempos varia bastante entre os grupos: fica abaixo de 10 % da média em 22 dos 36 grupos. As maiores variações aparecem nos lotes muito curtos ($N = 100$, com menos de 1 ms), mas há exceções também com $N = 10.000$: na inserção crescente, a Rubro-Negra teve média de 24,89 ms com desvio de 4,93 ms, e na remoção de 30 %, a AVL teve 17,72 ms com desvio de 6,16 ms (uma das repetições foi bem mais lenta que as outras). Por isso, as diferenças de tempo devem ser lidas como tendência medida nesta máquina, e não como constante.

8.2 Inserção: altura, tempo e rotações

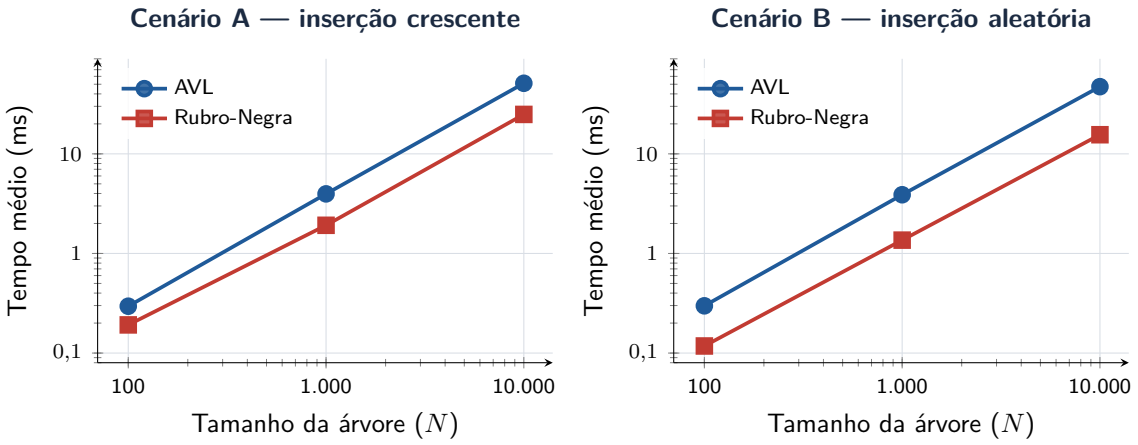


Figura 7. Tempo médio de inserção de N contas (escalas logarítmicas). Para $N = 10.000$, a AVL levou 2,1 vezes o tempo da Rubro-Negra no cenário A e 3,0 vezes no cenário B.

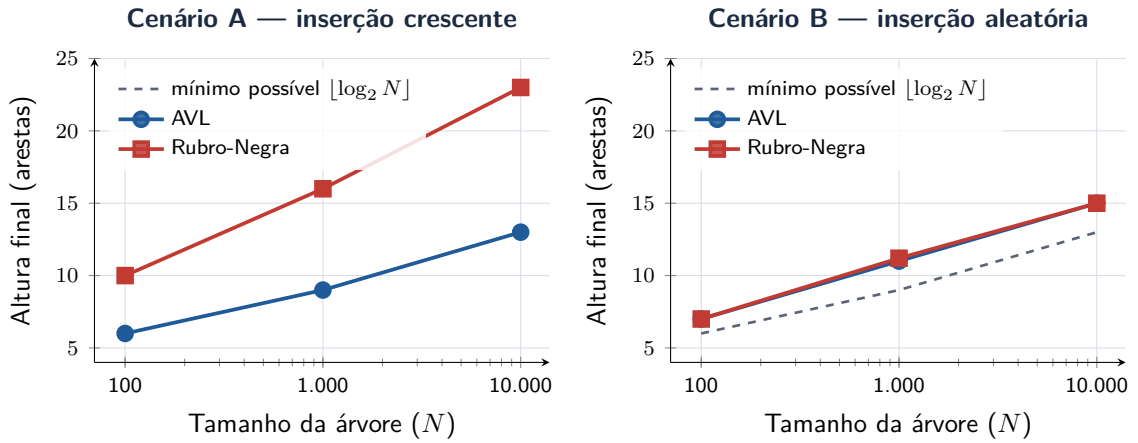


Figura 8. Altura final após a inserção. No cenário A, a AVL atinge exatamente o mínimo possível para N nós em todos os tamanhos (sua curva coincide com a linha tracejada); no cenário B, as duas curvas praticamente se sobrepõem.

A Tabela 15 detalha as rotações da AVL por caso de rebalanceamento, obtidas com `rotacoes_por_caso()` e as mesmas sementes da coleta (essa informação não é gravada nos CSVs). Em todas as linhas, a soma dos casos (LL e RR valendo 1 rotação, LR e RL valendo 2) coincide com as rotações do CSV oficial.

Tabela 15. Casos de rebalanceamento da AVL nos cenários A e B (média de 5 repetições).

Cenário	N	LL	RR	LR	RL	Rotações	Casos duplos
A	100	0	93	0	0	93	0,0 %
A	1.000	0	990	0	0	990	0,0 %
A	10.000	0	9.986	0	0	9.986	0,0 %
B	100	10,2	10,6	10,2	12,4	66	52,1 %
B	1.000	112	116,4	111,6	118,2	688	50,2 %
B	10.000	1.181,4	1.160,4	1.167,2	1.155,2	6.986,6	49,8 %

8.3 Busca

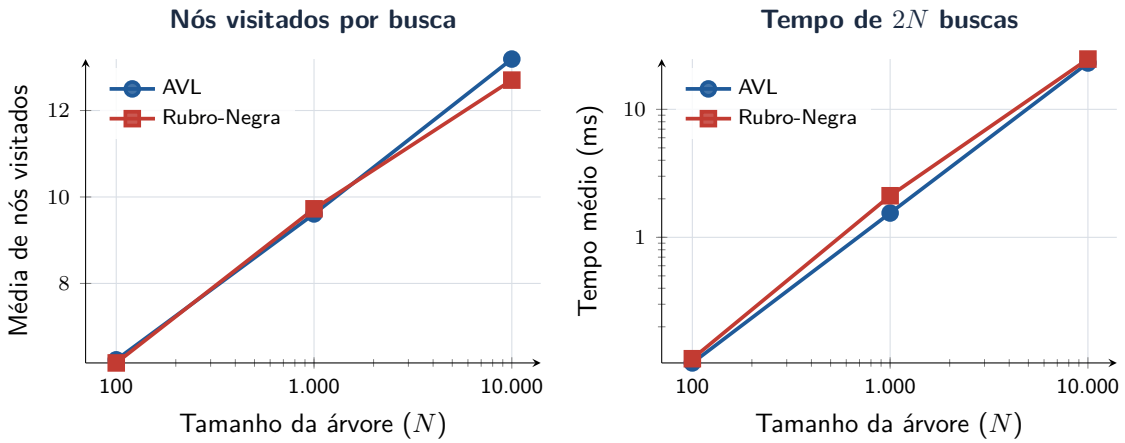


Figura 9. Cenário C. As árvores foram montadas por inserção aleatória (como no cenário B).

Tabela 16. Nós visitados por busca: chaves presentes × ausentes (`buscas.csv`, 5 repetições).

	Presentes		Ausentes		Máximo numa busca	
N	AVL	RN	AVL	RN	AVL	RN
100	5,87	5,92	6,60	6,40	8	8
1.000	9,22	9,25	10,00	10,20	12	13
10.000	12,58	12,61	13,80	12,80	16	16

8.4 Remoção

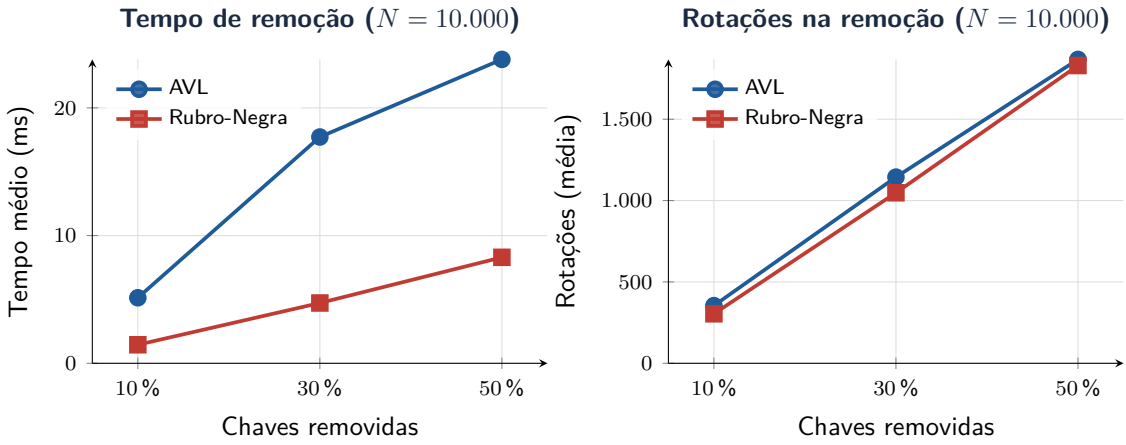


Figura 10. Cenário D. A base é reconstruída do zero para cada percentual.

Com 30 % das chaves removidas e $N = 10.000$, a altura média da AVL caiu de 15 para 14,4, enquanto a da Rubro-Negra permaneceu em 15; com 50 %, foram 14 e 14,2, respectivamente.

9 Discussão: respostas às perguntas do enunciado

Redação conjunta: I1, I2 e I3 · cada resposta separa o que foi **medido** do que é **expectativa teórica**.

9.1 Qual estrutura apresentou menor altura?

Resultado medido Na inserção crescente (A), a **AVL**, em todos os tamanhos: 6, 9 e 13, contra 10, 16 e 23 na Rubro-Negra. A altura da AVL é *exatamente* o mínimo possível para N nós, $\lfloor \log_2 N \rfloor$ (6, 9 e 13); a da Rubro-Negra ficou 77 % maior para $N = 10.000$. Na inserção aleatória (B), as alturas empataram para $N = 100$ e $N = 10.000$ (7 e 15 nas duas); para $N = 1.000$ a média foi 11 na AVL e 11,2 na Rubro-Negra (uma das cinco repetições da Rubro-Negra terminou com 12). Depois das remoções, a AVL ficou igual ou ligeiramente mais baixa (30 %: 14,4 contra 15; 50 %: 14 contra 14,2, com $N = 10.000$).

Limites teóricos conhecidos A AVL garante altura de no máximo cerca de $1{,}44 \log_2(N + 2)$, e a Rubro-Negra, $2 \log_2(N + 1)$; para $N = 10.000$, isso dá aproximadamente 19,1 e 26,6. As alturas medidas respeitam os dois limites. A inserção crescente levou a Rubro-Negra a 23, perto do seu limite, enquanto a AVL ficou no mínimo absoluto: o critério de balanceamento da AVL é mais rígido.

9.2 Qual estrutura realizou mais rotações?

Resultado medido

- **Inserção crescente (A):** praticamente o mesmo número: 9.986 na AVL e 9.976 na Rubro-Negra, cerca de uma rotação por inserção nas duas.
- **Inserção aleatória (B):** a AVL fez mais: 6.986,6 contra 5.788,8 (0,70 contra 0,58 por inserção). Nos três tamanhos a AVL fez mais rotações nesse cenário.
- **Remoção ($N = 10.000$):** a AVL fez mais com 10 % (354,4 × 304,2) e 30 % (1.144,2 × 1.048); com 50 % ficaram próximas (1.867,6 × 1.830).

A vantagem da Rubro-Negra em rotações *não* é universal nos tamanhos menores: com $N = 100$ e 50 % removidos, ela fez 19,4 rotações contra 17,6 da AVL; com $N = 1.000$ e 10 %, 31 contra 29,8.

9.3 Como as recolorações da Rubro-Negra aparecem nos experimentos?

Resultado medido Elas são a operação de rebalanceamento mais frequente: de 4 a 5 por rotação em todos os cenários com $N = 10.000$ (Tabela 10). O custo depende da ordem de entrada: 4,99 recolorações por inserção crescente contra 2,31 por inserção aleatória. Na remoção, o custo por operação sobe de 1,40 (10 %) para 1,78 (50 %). O caso tio vermelho, que recolore sem rotacionar, responde por 60 % das recolorações na inserção crescente e 67 % na aleatória. Nas buscas (C), não há recoloração. A decomposição completa está na Seção 6.7.

9.4 A menor altura resultou sempre em menor tempo de execução?

Resultado medido **Não.** Na inserção crescente, a AVL terminou muito mais baixa (13 contra 23) e, mesmo assim, levou 2,1 vezes o tempo da Rubro-Negra (51,12 contra 24,89 ms). Na busca, as árvores tinham a mesma altura (15) e a Rubro-Negra visitou *menos* nós por busca (12,70 contra 13,19), mas a AVL foi 7 % mais rápida (23,02 contra 24,71 ms). Portanto, nem a altura nem o número de nós visitados explicam, sozinhos, os tempos medidos.

Hipóteses para os tempos — não testadas As duas implementações diferem em estilo: na AVL, inserção e remoção são recursivas, com uma chamada de função por nível e recálculo de altura em cada ancestral; na Rubro-Negra, são iterativas. Em Python, chamadas de função são relativamente caras, o que pode explicar parte da vantagem da Rubro-Negra na inserção e na remoção. Na busca, iterativa nas duas, a Rubro-Negra compara cada nó com o atributo `self.NIL`, enquanto a AVL compara com a constante `None`; essa diferença pode pesar no laço mais executado. Nenhuma dessas hipóteses foi isolada por um experimento.

9.5 Como a ordem de inserção influenciou as duas estruturas?

Resultado medido A ordem teve **efeitos opostos na altura**. A entrada crescente, que seria o pior caso de uma árvore binária de busca sem balanceamento, deixou a **AVL** *mais baixa* que a entrada aleatória (13 contra 15), e a **Rubro-Negra** bem *mais alta* (23 contra 15).

As duas fizeram mais trabalho de rebalanceamento na entrada crescente. A AVL fez 9.986 rotações, contra 6.986,6 na aleatória (30 % menos na aleatória), e usou só o caso RR (Tabela 15), porque toda chave nova entra no extremo direito; na aleatória, os quatro casos aparecem em frequências parecidas, e cerca de metade dos rebalanceamentos são duplos. A Rubro-Negra fez 4,99 recolorações por inserção na crescente contra 2,31 na aleatória.
No tempo, a ordem pesou mais na Rubro-Negra: 24,89 contra 15,56 ms (60 % a mais na crescente), contra 51,12 e 47,28 ms na AVL (8 % a mais).

9.6 Em quais operações as diferenças foram mais evidentes?

Resultado medido **Maiores diferenças:** o tempo de remoção, em que a AVL levou 3,5, 3,8 e 2,9 vezes o tempo da Rubro-Negra (10 %, 30 % e 50 %, com $N = 10.000$); o tempo de inserção aleatória (3,0 vezes) e crescente (2,1 vezes); e a altura na inserção crescente (13 contra 23).
Menores diferenças: o número de nós visitados por busca (diferença de no máximo 3,8 % em todos os tamanhos), a altura na inserção aleatória (empate) e as rotações na inserção crescente (diferença de 10 em quase 10.000). Na busca, foi a AVL que levou vantagem de tempo, de 7 % com $N = 10.000$ e 27 % com $N = 1.000$.

9.7 Quais limitações metodológicas existem?

1. **Estilo de implementação.** AVL recursiva e Rubro-Negra iterativa: as comparações de tempo medem as duas *implementações*, e não apenas as duas estruturas. É o principal fator de confusão do experimento.
2. **Uma única máquina e um único interpretador** (CPython 3.13.2, Windows 11); os tempos absolutos e até a ordem entre as árvores podem mudar em outro ambiente.
3. **Cinco repetições, sem teste estatístico.** Alguns grupos têm desvio alto (Seção 8); diferenças pequenas de tempo não devem ser generalizadas.
4. **Ruído do Python.** Interpretador e coletor de lixo introduzem variações que podem mascarar diferenças pequenas, sobretudo nos lotes de menos de 1 ms ($N = 100$).
5. **Tempo por lote.** Mede-se o tempo total de cada lote, não o de cada operação; por isso não é possível atribuir o tempo a rotações ou a recolorações.
6. **Busca só em árvores aleatórias.** O cenário C usa árvores do cenário B, em que as alturas empataram; o efeito da altura maior da Rubro-Negra na entrada crescente sobre a busca não foi medido.
7. **Três tamanhos.** Apenas $N = 100$, 1.000 e 10.000.

10 Conclusão

As duas árvores foram implementadas com as mesmas convenções de contagem, passaram em 158 testes automatizados e terminaram válidas nas 180 medições, cujos valores determinísticos foram reproduzidos de forma independente. Nos dados medidos, a **AVL** manteve a árvore mais compacta: atingiu a altura mínima possível na entrada ordenada (13 contra 23 para $N = 10.000$) e empatou na entrada aleatória. A **Rubro-Negra** fez menos rotações em 13 dos 15 grupos de inserção e remoção e foi mais rápida em todos eles (com $N = 10.000$, de 2,1 a 3,8 vezes), com 4 a 5 recolorações por rotação. Nas buscas, com a mesma altura, as duas visitaram praticamente o mesmo número de nós, e a AVL foi ligeiramente mais rápida.

A comparação também mostrou que menor altura não implica menor tempo nesta implementação, e que parte das diferenças de tempo pode vir do estilo de cada código (recursivo ou iterativo), e não só da estrutura. Essas leituras valem para as condições do experimento (uma máquina, Python, cinco repetições, três tamanhos) e devem ser reforçadas com mais repetições e implementações no mesmo estilo antes de serem generalizadas.

Apêndice A — Resultados completos

Médias das 5 repetições, calculadas a partir de `resultados/resultados_brutos.csv`. O tempo está em milissegundos, com o desvio-padrão amostral entre parênteses. No cenário D, a altura aparece como "antes → depois". "Nós/busca" só se aplica ao cenário C, e recolorações não se aplicam à AVL.

Tabela 17. Resultados por cenário, tamanho e estrutura.

Cenário	N	Estrutura	Tempo (ms)		Altura	Rotações	Recolorações	Nós/busca
A	100	AVL	0,295	(0,015)	6	93	—	—
A	100	Rubro-Negra	0,192	(0,083)	10	89	445	—
A	1.000	AVL	3,96	(0,247)	9	990	—	—
A	1.000	Rubro-Negra	1,92	(0,165)	16	983	4.909	—
A	10.000	AVL	51,12	(1,67)	13	9.986	—	—
A	10.000	Rubro-Negra	24,89	(4,93)	23	9.976	49.877	—
B	100	AVL	0,298	(0,105)	7	66	—	—
B	100	Rubro-Negra	0,118	(0,027)	7	58	223,6	—
B	1.000	AVL	3,89	(0,390)	11	688	—	—
B	1.000	Rubro-Negra	1,36	(0,282)	11,2	581,2	2.295,8	—
B	10.000	AVL	47,28	(0,863)	15	6.986,6	—	—
B	10.000	Rubro-Negra	15,56	(0,575)	15	5.788,8	23.096,8	—
C	100	AVL	0,104	(0,009)	7	0	—	6,23
C	100	Rubro-Negra	0,113	(0,004)	7	0	0	6,16
C	1.000	AVL	1,55	(0,071)	11	0	—	9,61
C	1.000	Rubro-Negra	2,11	(0,425)	11,2	0	0	9,73
C	10.000	AVL	23,02	(0,439)	15	0	—	13,19
C	10.000	Rubro-Negra	24,71	(1,65)	15	0	0	12,70
D 10 %	100	AVL	0,053	(0,063)	7 → 7	2,4	—	—
D 10 %	100	Rubro-Negra	0,014	(0,002)	7 → 7	2,2	12,8	—
D 10 %	1.000	AVL	0,342	(0,023)	11 → 11	29,8	—	—
D 10 %	1.000	Rubro-Negra	0,127	(0,020)	11,2 → 11	31	141,2	—
D 10 %	10.000	AVL	5,13	(1,04)	15 → 15	354,4	—	—
D 10 %	10.000	Rubro-Negra	1,45	(0,064)	15 → 15	304,2	1.399	—
D 30 %	100	AVL	0,069	(0,006)	7 → 6,4	10,6	—	—
D 30 %	100	Rubro-Negra	0,035	(0,007)	7 → 6,8	8,2	46,8	—
D 30 %	1.000	AVL	1,05	(0,077)	11 → 10,6	104,8	—	—
D 30 %	1.000	Rubro-Negra	0,357	(0,014)	11,2 → 10,8	99,4	490,6	—
D 30 %	10.000	AVL	17,72	(6,16)	15 → 14,4	1.144,2	—	—
D 30 %	10.000	Rubro-Negra	4,72	(0,661)	15 → 15	1.048	4.893,8	—
D 50 %	100	AVL	0,110	(0,008)	7 → 6	17,6	—	—
D 50 %	100	Rubro-Negra	0,055	(0,003)	7 → 6	19,4	94	—

Tabela 17 — continuação

Cenário	N	Estrutura	Tempo (ms)		Altura	Rotações	Recolorações	Nós/busca
D 50 %	1.000	AVL	1,63	(0,072)	11 → 10	182,6	—	—
D 50 %	1.000	Rubro-Negra	0,591	(0,011)	11,2 → 10,2	182,2	895	—
D 50 %	10.000	AVL	23,80	(0,451)	15 → 14	1.867,6	—	—
D 50 %	10.000	Rubro-Negra	8,30	(0,587)	15 → 14,2	1.830	8.902,4	—

Apêndice B — Testes e reprodução

B.1 Suíte de testes

A suíte usa `unittest` (biblioteca padrão) e roda com `python -m unittest discover -s testes -v`. Na versão entregue, os 158 testes passam, inclusive num ambiente virtual novo, sem nenhuma dependência instalada.

Tabela 18. Testes por arquivo.

Arquivo	Testes	Autor	Cobertura
`test_avl.py`	50	I1	rotações, inserção, busca, remoção, validação, exibição, casos, modo detalhado
`test_experimentos_insercao.py`	12	I1	sementes, mesmas listas, ordem alternada, cronômetro limpo, colunas do CSV
`test_rubro_negra.py`	66	I2	casos de inserção e remoção com espelhos, bordas, validador, 2.000 operações aleatórias
`test_modelos.py`	8	I3	normalização e rejeição de cada campo, imutabilidade
`test_integracao.py`	22	I3	coleção comum, duplicata, entradas inválidas, divergência entre árvores, menu, executor
Total	**158**		

B.2 Como reproduzir

Preparar o ambiente	`python -m venv .venv` e ativar; `pip install -r requirements.txt`
Rodar os testes	`python -m unittest discover -s testes -v`
Coletar os dados	`python experimentos.py`
Gerar resumo e gráficos	`python gerar_graficos.py`
Usar o menu	`python main.py`
Demonstração da AVL	`python demo_avl.py -pausar`

Os arquivos gerados ficam em `resultados/`: `resultados_brutos.csv` (uma linha por medição); `buscas.csv` (uma linha por busca do cenário C); `resumo_medias.csv` (médias e desvios-padrão); `config_experimentos.json` (parâmetros, sementes e ambiente); e os gráficos `grafico_*.png`. Com as mesmas sementes, altura, rotações, recolorações e nós visitados se repetem exatamente; o tempo depende da máquina.